JobSheet - Week 7 – Multi-Thread

Nama: Muhammad Fadhil Arifin

NIM: 251524076

Kelas: 1C

Repo GitHub: [Tautan Repo GitHub]

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W7_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W7_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Transfer Fulus

Soal Praktikum

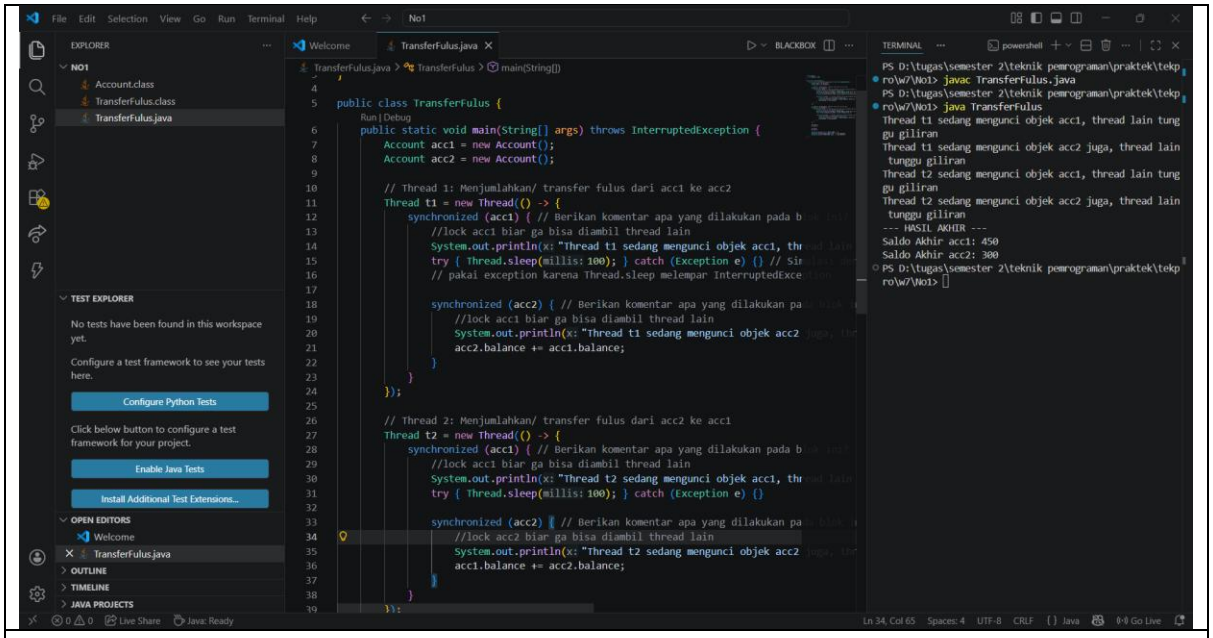
Instruksi Pengerjaan:
a. **Analisis Masalah**: Perhatikan source code di bawah ini, masalah apa yang Anda temukan? Berikan penjelasannya pada setiap bagian source code dengan menyeluruh berdasarkan materi yang telah diperoleh pada kelas teori!
b. **Solusi Terhadap Masalah**: Tuliskan solusi berupa source code dan penjelasan solusi terhadap masalah tersebut! Berikan penjelasannya pada setiap bagian source code!
 	
Source Code:
class Account {
  int balance = 150;
}

```
public class TransferFulus {
  public static void main(String[] args) throws InterruptedException {
    Account acc1 = new Account();
    Account acc2 = new Account();

    // Thread 1: Menjumlahkan/ transfer fulus dari acc1 ke acc2
    Thread t1 = new Thread(() -> {
      synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
        System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
        try { Thread.sleep(100); } catch (Exception e) {} // Simulasi dengan
memberikan jeda. Mengapa diperlukan Exception?

        synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
          System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
          acc2.balance += acc1.balance;
        }
      }
    });

    // Thread 2: Menjumlahkan/ transfer fulus dari acc2 ke acc1
    Thread t2 = new Thread(() -> {
      synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
        System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
        try { Thread.sleep(100); } catch (Exception e) {}

        synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
          System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
          acc1.balance += acc2.balance;
        }
      }
    });

    t1.start();
    t2.start();

    t1.join();
    t2.join();

    System.out.println("--- HASIL AKHIR ---");
    System.out.println("Saldo Akhir acc1: " + acc1.balance);
                System.out.println("Saldo Akhir acc2: " + acc2.balance);
  }
}
```

Source Code

```
class Account {
    int balance = 150;
}

public class TransferFulus {
    public static void main(String[] args) throws InterruptedException {
        Account acc1 = new Account();
        Account acc2 = new Account();

        // Thread 1: Menjumlahkan/ transfer fulus dari acc1 ke acc2
        Thread t1 = new Thread(() -> {
            synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok
ini?
                //lock acc1 biar ga bisa diambil thread lain
                 System.out.println("Thread t1 sedang mengunci objek acc1, thread
lain tunggu giliran");
                  try { Thread.sleep(100); } catch (Exception e) {} // Simulasi
dengan memberikan jeda. Mengapa diperlukan Exception?
               // pakai exception karena Thread.sleep melempar InterruptedException

                synchronized (acc2) { // Berikan komentar apa yang dilakukan pada
blok ini?
                    //lock acc1 biar ga bisa diambil thread lain
                    System.out.println("Thread t1 sedang mengunci objek acc2 juga,
thread lain tunggu giliran");
                    acc2.balance += acc1.balance;
                }
            }
        });

        // Thread 2: Menjumlahkan/ transfer fulus dari acc2 ke acc1
        Thread t2 = new Thread(() -> {
            synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok
ini?
                //lock acc1 biar ga bisa diambil thread lain
                 System.out.println("Thread t2 sedang mengunci objek acc1, thread
lain tunggu giliran");
                try { Thread.sleep(100); } catch (Exception e) {}

                synchronized (acc2) { // Berikan komentar apa yang dilakukan pada
blok ini?
                    //lock acc2 biar ga bisa diambil thread lain
                    System.out.println("Thread t2 sedang mengunci objek acc2 juga,
thread lain tunggu giliran");
                    acc1.balance += acc2.balance;
                }
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("--- HASIL AKHIR ---");
        System.out.println("Saldo Akhir acc1: " + acc1.balance);
        System.out.println("Saldo Akhir acc2: " + acc2.balance);
    }
}
```

Screenshot Hasil



Penjelasan

Masalahnya karena t1 ngunci acc1 dan t2 ngunci acc 2. Jadi ketika t1 mau ngabil acc 2 harus nunggu t2 lepas dulu tapi t2 ga bakal lepas acc2 karena dia juga nunggu t1 buat lepas acc1. Keadaan saling menuggu atau deadlock ini membuat program ga bisa jalan.

Solusinya urutan pengambilan t1 dan t2 harus sama sehingga ketika salah satu nya mengunci acc1, yang lain tidak mengunci acc 2 tapi mengantri untuk mengambil acc1. Akibatnya thread pertama yang dapat acc 1 dapat mengambil acc 2 untuk menyelesaikan tugas dan kemudian melepaskan kedua acc agar bisa diambil oleh thread lain.

Nama Teman Hal yang Dibantu (Opsional)

No. 2 Resto Ayam

Soal Praktikum
Instruksi Pengerjaan: a. **Analisis Masalah**: Perhatikan source code di bawah ini, masalah apa yang Anda temukan? Berikan penjelasannya pada setiap bagian source code dengan menyeluruh berdasarkan materi yang telah diperoleh pada kelas teori! b. **Solusi Terhadap Masalah**: Tuliskan solusi berupa source code dan penjelasan solusi terhadap masalah tersebut! Berikan penjelasannya pada setiap bagian source code! **Source Code:**

```
class Resto {
  private int chickenStock = 100;

  public void serveCustomer(String cashierName) {
    if (chickenStock > 0) {
      try { Thread.sleep(10); } catch (InterruptedException e) {}

      chickenStock--;
      System.out.println(cashierName + " berhasil menjual 1 ayam. Sisa stok: " +
chickenStock);
    } else {
      System.out.println(cashierName + " gagal: Stok Habis!");
    }
  }

  public int getRemainingStock() {
    return chickenStock;
  }
}

public class RestoSimulasi {
  public static void main(String[] args) throws InterruptedException {
    Resto ayamJuicyLuicyGallagher = new Resto();

    Runnable task = () -> {
      for (int i = 0; i < 40; i++) {

ayamJuicyLuicyGallagher.serveCustomer(Thread.currentThread().getName());
      }
    };

    Thread kasir1 = new Thread(task, "Kasir-A");
    Thread kasir2 = new Thread(task, "Kasir-B");
```

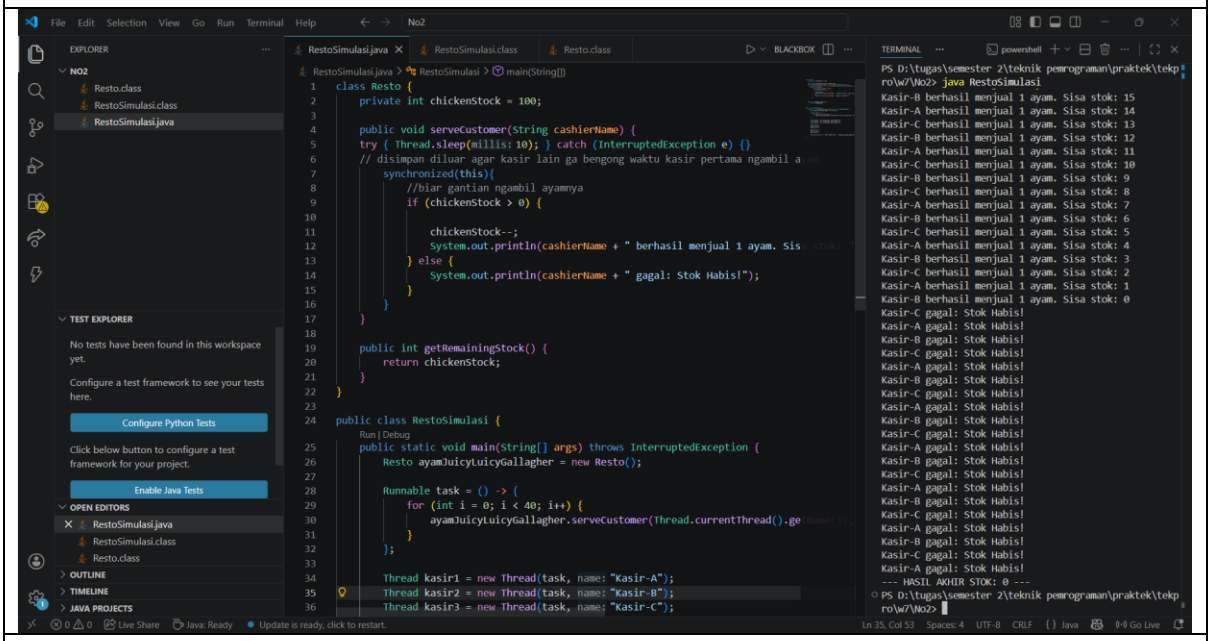
```
    Thread kasir3 = new Thread(task, "Kasir-C");

    kasir1.start();
    kasir2.start();
    kasir3.start();

    kasir1.join();
    kasir2.join();
    kasir3.join();

    System.out.println("--- HASIL AKHIR STOK: " +
ayamJuicyLuicyGallagher.getRemainingStock() + " ---");
  }
}
```

Source Code

```
class Resto {
    private int chickenStock = 100;

    public void serveCustomer(String cashierName) {
    try { Thread.sleep(10); } catch (InterruptedException e) {}
    // disimpan diluar agar kasir lain ga bengong waktu kasir pertama ngambil ayam
        synchronized(this){
            //biar gantian ngambil ayamnya
            if (chickenStock > 0) {

                chickenStock--;
                 System.out.println(cashierName + " berhasil menjual 1 ayam. Sisa
stok: " + chickenStock);
            } else {
                System.out.println(cashierName + " gagal: Stok Habis!");
            }
        }
    }

    public int getRemainingStock() {
        return chickenStock;
    }
}

public class RestoSimulasi {
    public static void main(String[] args) throws InterruptedException {
        Resto ayamJuicyLuicyGallagher = new Resto();

        Runnable task = () -> {
            for (int i = 0; i < 40; i++) {
                ayamJuicyLuicyGallagher.serveCustomer(Thread.currentThread().getN
ame());
            }
        };

        Thread kasir1 = new Thread(task, "Kasir-A");
        Thread kasir2 = new Thread(task, "Kasir-B");
        Thread kasir3 = new Thread(task, "Kasir-C");

        kasir1.start();
        kasir2.start();
        kasir3.start();

        kasir1.join();
        kasir2.join();
        kasir3.join();
```

```
                    System.out.println("---    HASIL    AKHIR    STOK:    "    +
ayamJuicyLuicyGallagher.getRemainingStock() + " ---");
    }
}
```

Screenshot Hasil



Penjelasan

Masalahnya adalah ketika ketiga kasir mau ngambil ayam bersamaan mereka lihat stok yang sama. Sehingga ketika stok ayam sisa satu ketiganya mengambil ayam bersamaan lagi hasilnya stok ayam akan jadi -2.
Solusinya dengan menggunakan synchronized agar para kasir mengantri untuk mengambil ayam sehingga ketika ayam sisa 1 Cuma satu kasir yang ambil ayam.
Sleep diletakan diluar agar ketiga kasir menjalankan sleep secara bersamaan.
Mereka baru antre saat mau mengambil ayamnya saja.

Nama Teman Hal yang Dibantu (Opsional)

Muhammad Zaky Al Askary (079): Memberi ide untuk naruh Thread.sleep(10) di luar blok synchronized biar proses nunggu/jeda kasirnya tetap jalan barengan (paralel).

No. 3 Penjumlahan Paralel

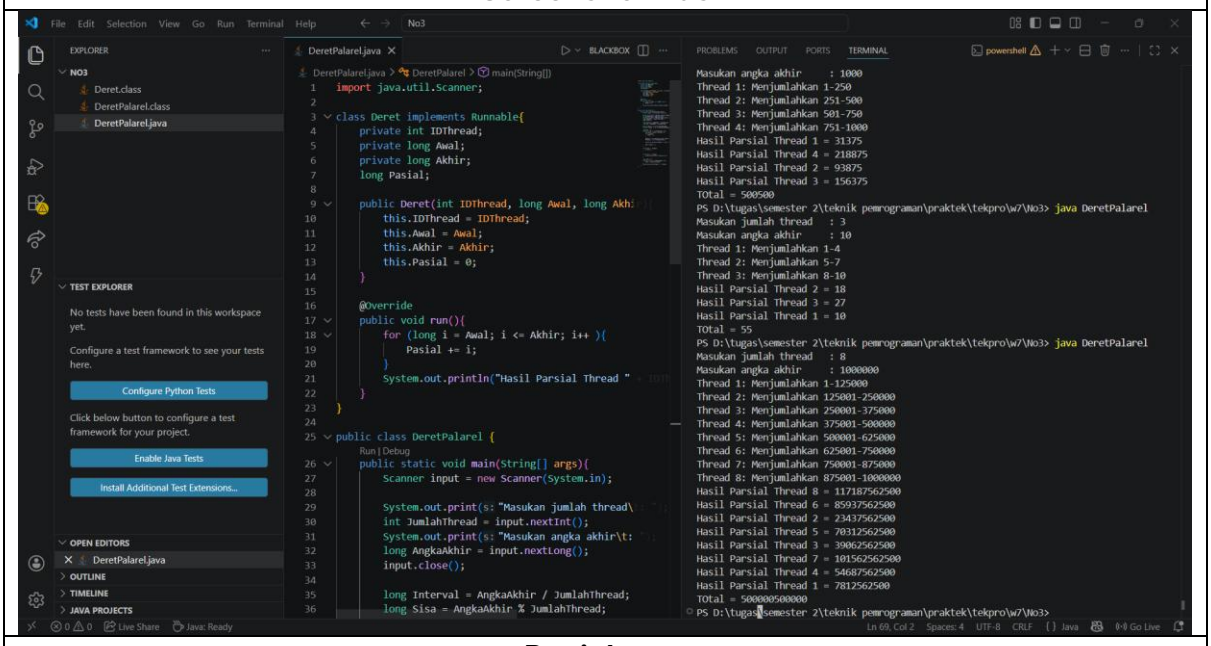
Soal Praktikum
Instruksi Pengerjaan: Buatlah suatu program yang bertujuan untuk melakukan penjumlahan secara paralel dengan ketentuan sebagai berikut: - Menerima Input: (1) Jumlah Thread; (2) Angka Akhir - Contoh: Menjumlahkan angka dari 1 sampai "Angka Akhir". Jika "Jumlah Thread" = 4, "Angka Akhir" = 1000, maka: - Thread 1: Menjumlahkan 1 - 250 - Thread 2: Menjumlahkan 251 - 500 - Thread 3: Menjumlahkan 501 - 750 - Thread 4: Menjumlahkan 751 - 1000 - Hasil akhir diperoleh dari hasil parsial (dari masing-masing thread) yang dijumlahkan menjadi total akhir. - Perhatikan Thread Safety, Mekanisme Pembagian Tugas (Divide and Conquer), dan Syncronization agar hasil akhir sesuai! - Pada program, tampilkan output: - Tugas apa yang dikerjakan oleh masing-masing thread - Hasil parsial dari masing-masing thread - Hasil akhir yang merupakan penjumlahan dari semua hasil parsial - Berikan penjelasan yang lengkap untuk setiap bagian pentingnya!

Source Code

```
import java.util.Scanner;

class Deret implements Runnable{
    private int IDThread;
    private long Awal;
    private long Akhir;
    long Pasial;

    public Deret(int IDThread, long Awal, long Akhir){
        this.IDThread = IDThread;
        this.Awal = Awal;
        this.Akhir = Akhir;
        this.Pasial = 0;
    }

    @Override
    public void run(){
        for (long i = Awal; i <= Akhir; i++ ){
            Pasial += i;
        }
        System.out.println("Hasil Parsial Thread " + IDThread + " = " + Pasial);
    }
}

public class DeretPalarel {
    public static void main(String[] args){
        Scanner input = new Scanner(System.in);

        System.out.print("Masukan jumlah thread\t: ");
        int JumlahThread = input.nextInt();
        System.out.print("Masukan angka akhir\t: ");
        long AngkaAkhir = input.nextLong();
        input.close();

        long Interval = AngkaAkhir / JumlahThread;
```

```
        long Sisa = AngkaAkhir % JumlahThread;

        Deret[] Deret = new Deret[JumlahThread];
        Thread[] Thread  = new Thread[JumlahThread];

        long Awal = 1;
        for(int i = 0; i < JumlahThread; i++){
            long Akhir = Awal + Interval -1;
            if(i < Sisa){
                Akhir += 1;
            }

            Deret[i] = new Deret(i+1, Awal, Akhir);
            Thread[i] = new Thread(Deret[i]);

            System.out.println("Thread " + (i+1) + ": Menjumlahkan " + Awal + "-"
+ Akhir);

            Awal = Akhir + 1;
        }

        for(Thread t : Thread){
            t.start();
        }


        for(Thread t : Thread){
            try{t.join();}catch (Exception e) {}
        }

        long Total = 0;
        for(int i = 0; i < JumlahThread; i++){
            Total += Deret[i].Pasial;
        }
        System.out.println("TOtal = " + Total);
    }
}
```

Screenshot Hasil



Penjelasan

Pembagian tugas (Divide and Conquer) dibuat dengan membagi angka akhir dengan jumlah thread, kalau ada sisa bagi ditaruh di thread awal biar pembagiannya adil. Kodenya udah aman dari tabrakan data meskipun ga pakai synchronized, karena tiap

thread nyatet hasil hitungannya di variabel Pasial punya mereka masing-masing, jadi ga ada rebutan data. Terakhir, sinkronisasinya pakai t.join() biar program utamanya nungguin semua thread beres ngitung dulu. Kalau ga ditungguin, ntar programnya keburu nyetak total akhir padahal threadnya masih pada sibuk ngitung.
Nama Teman Hal yang Dibantu (Opsional)